T22: inlining the carry-less multiply of the u128 field (n = 73 scan gate)

Class: **engineering**, observation only. Same 4 vCPU Xeon 2.80 GHz KVM guest as T20 and T21 (23 isolated runs, 0 flagged contended; scan on cores 1 to 3). Protocol frozen before timing in `protocol.json` (commit d77fd50c5, new binary sha256 9988639a…, old binary ea4faa51…). Raw records `runs/`, derived values `reports/analysis.json` (`examples/t22_scan_analysis.rs`). Stacked on PRs #2329 and #2318.

Question, change, frozen rule

The T21 card hypothesised that `Gf2_128::reduce_fold` spends most of its instructions moving operands between general and vector registers around its carry-less products. A read of the disassembly (first-`ret` slice of the T21-head binary, 171 instructions) showed three out-of-line calls to `clmul_u64` per reduction, six push/pop pairs and no `pclmulqdq` inside the function: `clmul_u64` is a `#[target_feature(enable = "pclmulqdq")]` function without `#[inline]`. The change is one attribute, `#[inline]` on the x86_64 `clmul_u64` (`src/cryptanalysis/semaev_decomp.rs`; aarch64 unchanged and unmeasured). The 31 `semaev_decomp` unit tests pass, including the fold-versus-table equality tests for the tested n between 65 and 127.

Frozen gate (as T21): the 20,000-state scan at the real K = 600 base clears only if the median ns-per-probe ratio new/old is at most 0.8333 and its 95% bootstrap upper bound at most 0.90; the end-to-end stage runs only if it clears. 7 rounds, arm order rotated, an A/A arm (the old binary twice). Two builds, not one binary with a switch (inlining cannot be toggled at run time): build-to-build code layout is a confound the A/A arm does not measure.

Result: scan gate

arm	median ns/probe	ratio to old (95%)	speedup vs old
old	178.7	1	1
new (inlined clmul)	173.4	0.970 (0.913 to 1.009)	1.03 (0.99 to 1.10)
old2 (A/A)	188.0	1.052 (1.019 to 1.080)	0.95 (0.93 to 0.98)

Probes and hits are identical in all 21 runs. **The gate is not cleared** (0.970 against 0.8333; even the upper end of the speedup interval, 1.10, is below 1.2x), so the end-to-end stage was **not run**. **Validity flag:** the A/A interval excludes 1 (the same binary run twice differs by 5% in this session), so the scan is noise-limited at about 5% and the 3% difference between the builds is not resolved either way.

Deterministic instruction counts (callgrind, K = 40, 2000 states)

Collected Ir 2,631.6 M (old) to 2,232.4 M (new): **−399.2 M, −15.2%**, deterministic. By function: `reduce_fold` 424.1 M to 45.6 M, `mul_fold` 244.3 M to 439.6 M (the inlined products land in it), `clmul_u64` 216.0 M to 0; together 884.4 M to 485.2 M (**−45.1%**). `extract128` (424.4 M), `RootBatch128::solve` (252.2 M), `Gf2_128::sqr` (73.4 M) and `build_index128_impl` (825.0 M) are unchanged.

What it means

The change removes 45% of the instructions of the multiply and reduce path and 15% of the whole K = 40 profile, and the scan time does not move beyond a few percent that this run cannot resolve. So at K = 600 the probe loop is not limited by instruction count. Hypothesis (not measured here; this guest has no PMU): it is limited by latency, either of dependent chains (the batched inversion's prefix and back-substitution products are serial, 256 multiplications long per batch) or of cache and memory misses on the Bloom filter and the table. That also explains T21: cuts that save instructions without shortening a dependent chain or a miss do not show up in ns per probe.

The attribute is correct and harmless (equality tests pass, fewer instructions); it ships on the default path as an instruction-count improvement of the field multiply, not as a speed claim.

Limits

- Two builds, not one binary: layout confound not covered by the A/A arm; the callgrind Ir is the guard and is deterministic.
- The A/A interval excludes 1: the scan resolves about 5% in this session, so a gain below that would not be seen.
- x86_64 only; aarch64 (`PMULL`) not changed or measured. No end-to-end, no S claim (counted S identical by construction).
- The disassembly slice used for the hypothesis ends at the first `ret`; it is a code reading, not a result.

Next (T24 card, `NEXT_T24.md`)

Test the latency hypothesis directly with a same-binary switch: run the batched inversion as 4 interleaved Montgomery sub-batches (independent chains, more instruction-level parallelism, same results) and gate it on the same scan, with a per-instruction-count guard; and, separately, a prefetch-depth sweep for the Bloom line. Kill under 1.2x on the scan gate or any change in relations.

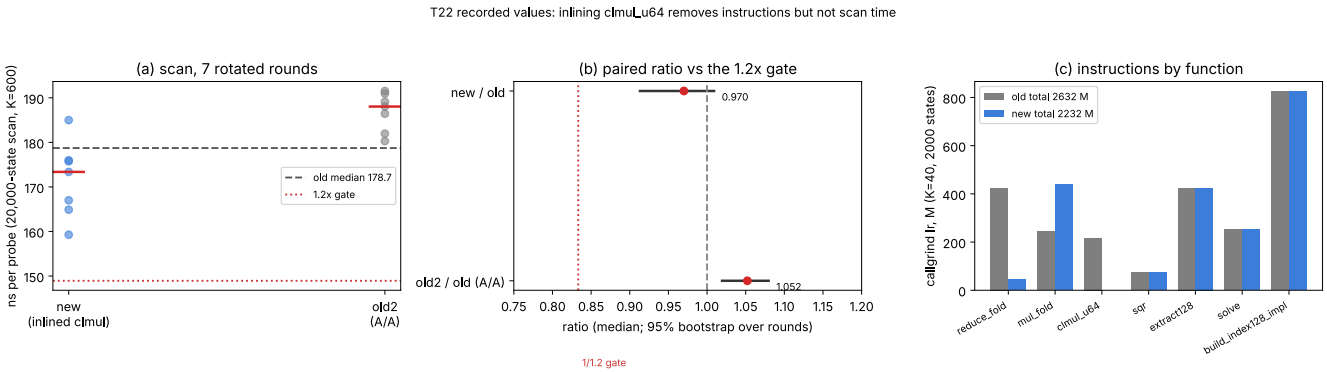


Figure 1: T22 recorded values: (a) scan ns per probe by round, (b) paired ratio against the 1.2x gate with the A/A arm, (c) callgrind instructions by function, old versus new. Source `reports/analysis.json`, `runs/`; plotted by `make_graph.py`.